Training a small GPT on TinyStories under a one-hour budget: a compute-optimal sweet spot

Steffany Lara Kerem Ozemre Tonio Kowalke
Ahmed Haj Hassine
DSA4212, National University of Singapore

November 8, 2026

Group members: Steffany Lara (A0350565H), Kerem Ozemre (A0 [fill]), Tonio Kowalke (A0 [fill]), Ahmed Haj Hassine (A0 [fill])

Code repository: `https://github.com/KeremOzemre/Optimization-for-Large-Scale-Data`

Abstract

Under a fixed one-hour, from-scratch training budget on identical hardware, we train 20 nanoGPT language models on TinyStories: five model sizes (16–253M parameters) crossed with four training-data fractions (25/50/75/100% of the training split). Holding the tokenizer, optimizer, schedule, context length, and effective batch fixed, we find a clear compute-optimal sweet spot: performance improves steeply from `small` to `xl` (≈124M parameters) and then *reverses* at `xxl` (≈253M), because the largest model is so slow that it consumes roughly half as many tokens in the hour and stays undertrained. The data fraction is almost irrelevant: no run exhausts even 25% of the split within the budget, so these runs are compute-bound, not data-bound. Selecting by validation loss gives `xl_data50` (val loss 1.5025, perplexity 4.49); on the held-out test split it reaches loss 1.4846, perplexity 4.41, and 0.524 bits per byte.

1 Compute budget declaration

Every model is trained from *random weights* (nanoGPT initialisation; no pretrained weights, embeddings, or tokenizer fine-tuning) for exactly one hour (3600 s) of in-loop training wall-clock *per model*. Data preparation, the final validation pass, and checkpointing happen outside the timed loop. All 20 runs use the same hardware: a single **NVIDIA H100 NVL MIG 3g.47gb** slice in `bfloat16`. Equal wall-clock does *not* imply equal token exposure, and quantifying that gap is central to our result (§5).

2 Research question

We ask: *which model size and training-data fraction give the best language-modeling performance under a fixed one-hour budget on identical hardware?* We hold the tokenizer, optimizer, schedule, context length, and effective batch size fixed and vary only (a) model size and (b) the fraction of the training split used.

3 Experimental setup

- **Data:** `roneneldan/TinyStories`. Official train split for training; the official validation split is divided 80/20 into validation/test at the story level (seed 42). The test split is used *only* for the final comparison (§5.1).
- **Tokenizer:** GPT-2 (`tiktoken`), vocabulary 50,257, with an end-of-text token appended to every story. Because the tokenizer is fixed, per-token cross-entropy and perplexity are directly comparable across all our runs.
- **Model:** nanoGPT [3] GPT with tied input/output embeddings, causal self-attention, context length 256, and dropout 0.1.
- **Evaluation objective:** held-out cross-entropy on the test split, reported also as perplexity (exp of the loss) and bits per byte. These are appropriate because cross-entropy is exactly the language-modeling training objective measured on unseen text, perplexity is its interpretable form, and bits per byte normalises across tokenizers so our numbers are comparable to other work [1]. The story-level 80/20 split (seed 42) prevents any story leaking between validation and test.

Table 1: The five model sizes. Width is divisible by the head count; the context length (256) and dropout (0.1) are shared. Microbatch × accumulation is held at an *effective batch of 16 sequences* (4096 tokens) per optimizer step for every size.

Size	Layers	Width	Heads	Params (M)	Microbatch × accum.
`small`	4	256	4	16.08	16×1
`medium`	6	384	6	30.02	16×1
`large`	8	512	8	51.04	16×1
`xl`	12	768	12	123.75	8×2
`xxl`	16	1024	16	253.09	4×4

Data fractions: 25/50/75/100% of the training split, giving a 5×4 grid of 20 runs. **Optimizer/schedule:** AdamW (lr 3×10^{-4}, weight decay 0.1, betas 0.9/0.95, gradient clip 1.0), with a time-based 2% warmup then cosine decay to 10% of the peak learning rate.

4 Methodology and reproducibility

1. `python run.py prepare --config configs/experiments/<name>.yaml` builds (or reuses) the token cache for the requested data fraction.
2. `python create_experiments.py` generates the 20 configs under `configs/experiments/` from `configs/default.yaml`.
3. Each config is trained with `python run.py train --config ...`; per-run outputs (`run_config.json`, `metrics.csv`, `checkpoint.pt`) land in `results/<name>/`. All checkpoints and logs are mirrored to the Hugging Face Hub at `dsa4212-tinystories/tinystories-nanogpt`.
4. The model is selected by *validation* loss; only the selected model is then evaluated on *test*.

5. `python eval_hf.py` downloads every checkpoint and scores the full test split; `python analyze.py --runs-dir results` aggregates all runs into `analysis/` (tables and the figures reproduced below).

5 Results

Table 2 lists all 20 runs sorted by final validation loss (the selection metric). Three patterns stand out, visualised in Figures 1–3.

Table 2: All 20 runs, sorted by final validation loss (the selection metric). "Tokens" is the number of training tokens actually processed inside the one-hour budget. Test numbers are reported here for completeness but were *not* used for selection (§5.1).

Run	Params (M)	Data%	Tokens (M)	Val loss	Val ppl	Test ppl
`xl_data50`	123.75	50	273.9	**1.5025**	**4.49**	4.41
`xl_data100`	123.75	100	279.8	1.5068	4.51	4.40
`xl_data75`	123.75	75	269.1	1.5099	4.53	4.43
`xxl_data100`	253.09	100	143.7	1.5106	4.53	4.56
`xl_data25`	123.75	25	283.3	1.5118	4.54	4.45
`xxl_data75`	253.09	75	141.1	1.5125	4.54	4.59
`xxl_data50`	253.09	50	148.2	1.5155	4.55	4.56
`xxl_data25`	253.09	25	143.2	1.5233	4.59	4.61
`large_data100`	51.04	100	562.4	1.5460	4.69	4.66
`large_data75`	51.04	75	562.0	1.5466	4.70	4.67
`large_data50`	51.04	50	557.9	1.5479	4.70	4.69
`large_data25`	51.04	25	563.2	1.5509	4.72	4.70
`medium_data100`	30.02	100	734.6	1.6444	5.18	5.16
`medium_data75`	30.02	75	738.3	1.6462	5.19	5.17
`medium_data50`	30.02	50	737.3	1.6527	5.22	5.20
`medium_data25`	30.02	25	719.1	1.6558	5.24	5.21
`small_data75`	16.08	75	940.8	1.8383	6.29	6.28
`small_data100`	16.08	100	984.3	1.8386	6.29	6.26
`small_data50`	16.08	50	985.5	1.8421	6.31	6.29
`small_data25`	16.08	25	972.0	1.8452	6.33	6.31

(1) A compute-optimal sweet spot. Bigger helps with sharply diminishing returns and then reverses: validation loss falls $1.84 \to 1.64 \to 1.55 \to 1.50$ from `small` to `xl`, but the best `xxl` run (1.5106) is *worse* than the best `xl` run (1.5025). `xxl` has roughly twice the parameters of `xl` but runs at about half the throughput, so it sees only ∼144M tokens in the hour versus ∼274M for `xl` and stays undertrained (Fig. 3). This is the compute-optimal trade-off of Hoffmann et al. [2]: under a fixed compute budget there is an optimal model size, and growing past it wastes the budget on parameters that never see enough tokens.

(2) Data fraction is nearly irrelevant. Within each size the four data fractions differ by only ∼0.01 nats and not monotonically (Fig. 2, right). The reason is in Table 2: no run processes more than ∼985M tokens, and even the 25% subset of TinyStories contains far more tokens than that, so training (which samples windows with replacement) never exhausts even the smallest subset. These runs are **compute-bound, not data-bound**.

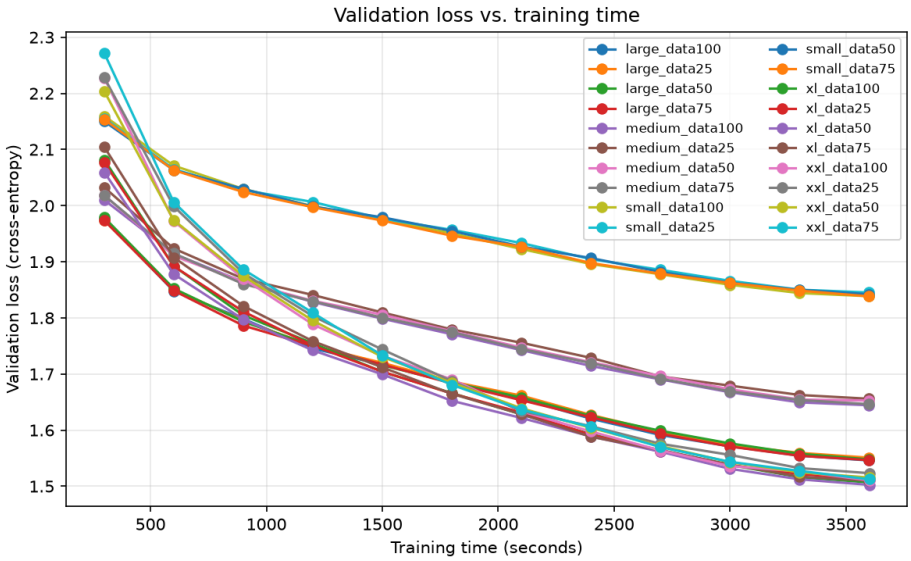


Figure 1: Validation loss over training time. Smaller models fall fastest early (they take many more steps per second) and plateau; `xl` and `xxl` start higher but overtake the smaller models and are still visibly improving at the one-hour mark, confirming they are compute-limited rather than converged.

(3) Test tracks validation. The test ranking mirrors validation (Table 2); the full-split test losses are slightly *lower* than validation because test is scored exhaustively over all 957,184 held-out tokens, whereas in-loop validation sampled a fixed number of windows.

5.1 Final test result

Selecting purely on validation loss gives `xl_data50` (val loss 1.5025, perplexity 4.49). Evaluated once on the untouched test split, it attains **test loss 1.4846, perplexity 4.41, and 0.5237 bits per byte**. We note the entire `xl` family is statistically indistinguishable (val 1.5025–1.5118; the 0.004 gap by which `data50` edges `data100` is within run-to-run noise), so the robust takeaway is *"`xl` (≈124M parameters) at any data fraction"* rather than 50% specifically.

5.2 Qualitative evaluation

Generated stories (temperature 0.8, top-k 40; full samples in the appendix) agree with the metrics but compress the differences. Even `small` produces grammatical, on-topic TinyStories; the gains with size are in *coherence over longer spans* — the `xl` winner maintains a single consistent plot and characters across a full paragraph, whereas `small` more often loops ("painted a dog, and a dog") or drops a thread. Qualitative quality thus supplements but does not replace the held-out metrics.

5.3 Estimating the compute-optimal size

Using only the trained runs, we estimate each run's training compute with the standard approximation $C \approx 6ND$ (N parameters, D tokens processed). Taking the best validation loss per size and fitting a parabola in $\log_{10}$(parameters) (Fig. 4, left) places the minimum — the compute-optimal size for a one-hour budget on this hardware — at $N^\star \approx 130$M parameters, essentially coinciding

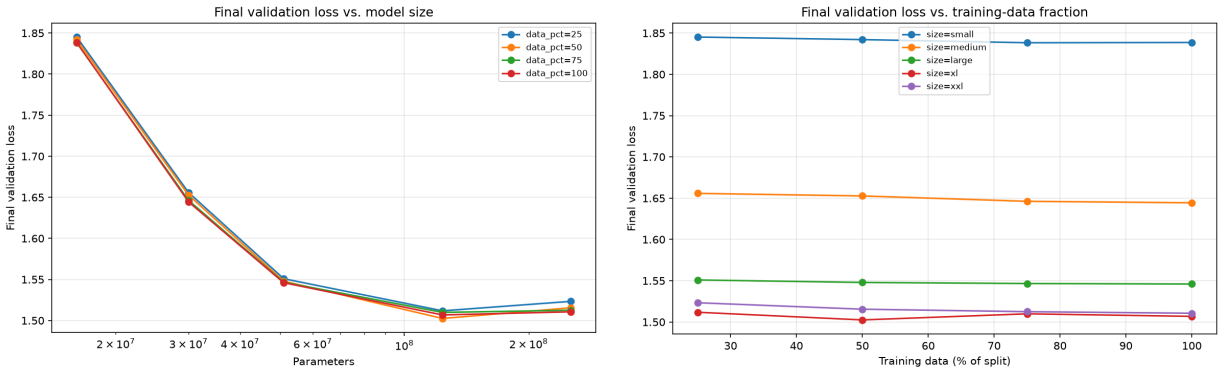


Figure 2: Left: final validation loss vs. model size (one line per data fraction). Loss drops steeply `small`→`medium`→`large`→`xl`, then ticks *up* at `xxl` — a sweet spot near 124M parameters. Right: loss vs. data fraction (one line per size). The lines are nearly flat; within every size the four fractions differ by only ∼0.01 nats and not even monotonically, i.e. the data fraction barely matters in this budget.

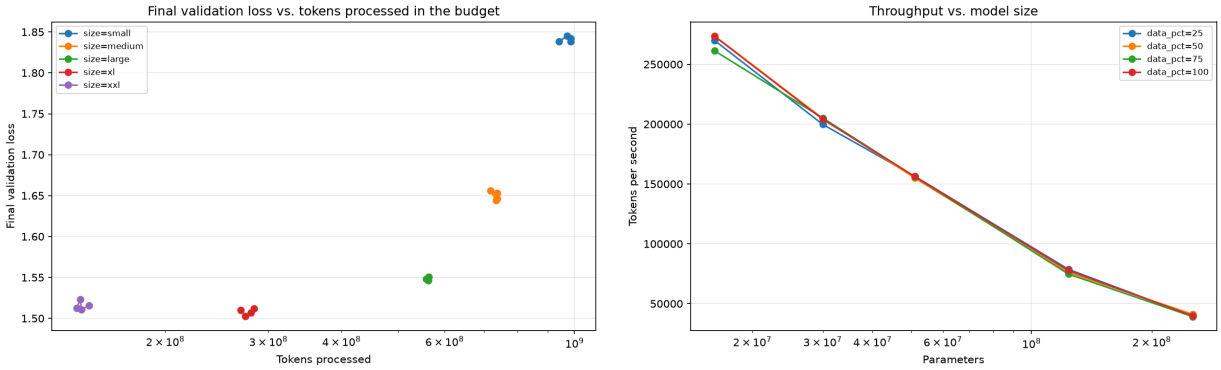


Figure 3: The core tension. Right: throughput collapses with size — from ∼270k tokens/s for `small` to ∼40k tokens/s for `xxl`. Left: consequently larger models process far fewer tokens in the same hour (∼985M for `small` vs ∼144M for `xxl`, a ∼7× gap), yet still reach lower loss per token — until `xxl`, where too few tokens finally outweigh the extra capacity.

with our empirical winner `xl` (124M). The loss–compute view (Fig. 4, right) exposes a secondary efficiency effect: although every run used the same wall-clock hour, the *delivered* compute grows with size ($\approx 9 \times 10^{16}$ FLOPs for `small` up to $\approx 2.2 \times 10^{17}$ for `xxl`), because the smallest models underutilise the GPU. `small` is therefore doubly penalised: it both lacks capacity and extracts less compute from the same hour.

6 Discussion

The winner wins because it sits at the compute-optimal point for this budget: large enough to have the capacity the smaller models lack, small enough to still see enough tokens in an hour to use that capacity. The experiment is firmly *compute-bound*: throughput, not data availability, is the binding constraint, which is exactly why the data fraction barely moves the loss and why `xxl` underperforms `xl`. With a larger budget we would expect the curve in Fig. 2 (left) to keep bending

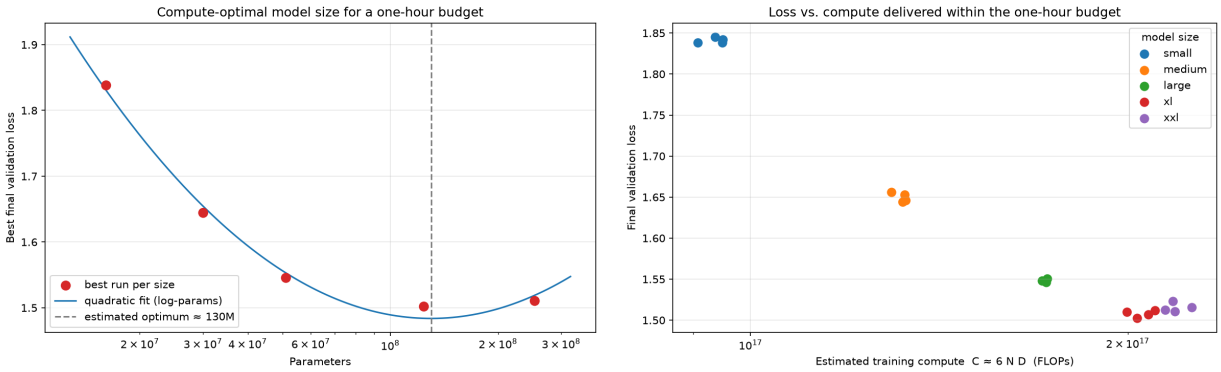


Figure 4: Left: best validation loss per size vs. parameters, with a quadratic fit in log-parameters whose vertex estimates the compute-optimal size ($N^\star \approx 130$M) for the one-hour budget — within noise of the empirical winner `xl`. Right: loss vs. estimated compute $C \approx 6ND$; the smallest models deliver the least compute despite the identical wall-clock budget. The fit uses five points and is indicative rather than precise.

right — `xxl` (and beyond) should eventually overtake `xl` once given enough tokens to train on, and the data fraction would start to matter once a run can consume a full subset. **Limitations:** training samples windows with replacement, so one hour guarantees neither a full epoch nor unique coverage; the single shared recipe (learning rate, warmup, batch) is not separately tuned per size and may slightly favour some sizes; validation used sampled windows rather than the exhaustive test-time pass; and the TinyStories source revision is not pinned.

7 Conclusion

Under a fixed one-hour, from-scratch budget on one H100 MIG slice, the best language model on TinyStories is `xl` (≈124M parameters); selected on validation, `xl_data50` reaches test perplexity 4.41 (0.524 bits/byte). The single most important trade-off is throughput versus capacity: because larger models process far fewer tokens in the same wall-clock time, "bigger" stops helping — and then hurts — well before the largest model we trained, while the amount of training data available makes almost no difference in this compute-bound regime.

References

[1] R. Eldan and Y. Li. *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv:2305.07759, 2023.

[2] J. Hoffmann et al. *Training Compute-Optimal Large Language Models* (Chinchilla). arXiv:2203.15556, 2022.

[3] A. Karpathy. *nanoGPT*. `https://github.com/karpathy/nanoGPT`, commit `3adf61e`, 2023.

A Generated story samples

The complete set of samples for all 20 runs (five prompts each) is in `analysis/generations_all.md`. As an illustration, both models continue "*Once upon a time*" coherently, but the `xl` winner sustains

a longer, more consistent narrative:

`small_data25`. *"Once upon a time, there was a little girl named Lily. Lily loved to play with her toys... She painted a picture of a flower and a house... Later that day, Lily's mom asked her to help. Lily was happy to help and started to decorate the room with pretty flowers."* (The thread loops back on itself.)

`xl_data50` **(selected).** *"Once upon a time, there was a little girl named Lily. Lily loved to play with her toys, especially her teddy bear. One day, Lily's mommy asked her to help decorate the house for a party... 'Mommy, can I carry my teddy bear?' she asked... They arrived at the living room and Lily's mommy gave her a big box of presents."* (One consistent, advancing plot.)